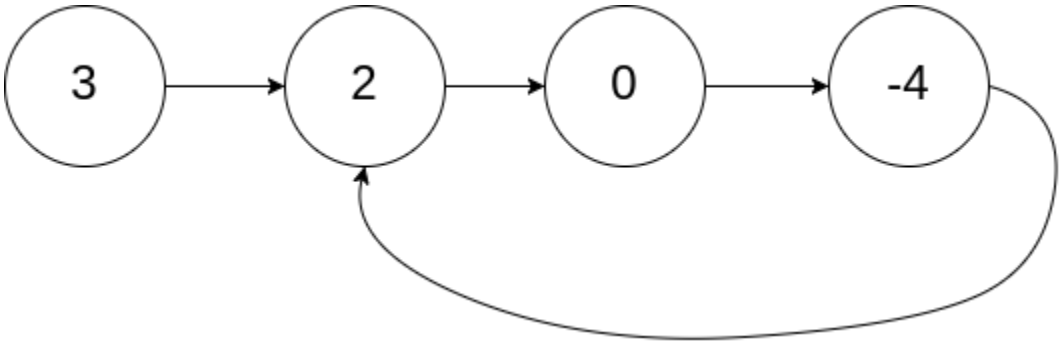
Set A

Q1. Given head, the head of a linked list, determine if the linked list has a cycle in it.
There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the next pointer. Internally, pos is used to denote the index of the node that tail's next pointer is connected to. Note that pos is not passed as a parameter. Return true if there is a cycle in the linked list. Otherwise, return false.

Example :



Input: head = [3,2,0,-4], pos = 1
Output: true

Explanation: There is a cycle in the linked list, where the tail connects to the 1st node (0-indexed).

One of the solution:

```
bool hasCycle(ListNode *head)
{
   if (head == nullptr || head->next == nullptr)
   {
      return false;  // Empty list or only one node → no cycle
   }

   ListNode *slow = head;
   ListNode *fast = head;
```

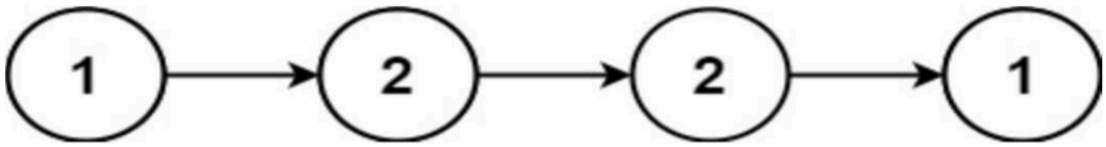
```
    while (fast != nullptr && fast->next != nullptr)
    {
        slow = slow->next;                  // move 1 step
        fast = fast->next->next;            // move 2 steps

        if (slow == fast)
        {
            return true; // cycle detected
        }
    }

    return false;                           // fast reached end → no cycle
}
```

****

Q2. Given the head of a singly linked list, return true if it is a palindrome or false otherwise.
Example :
Input:



Output: true
Function declaration: bool isPalindrome(ListNode* head) { }

One of the solution:

```
bool isPalindrome(ListNode* head)
{
    if (head == nullptr || head->next == nullptr)
    {
        return true;  // Empty list or single node is always palindrome
    }

    // Step 1: Find middle using slow and fast pointers
```

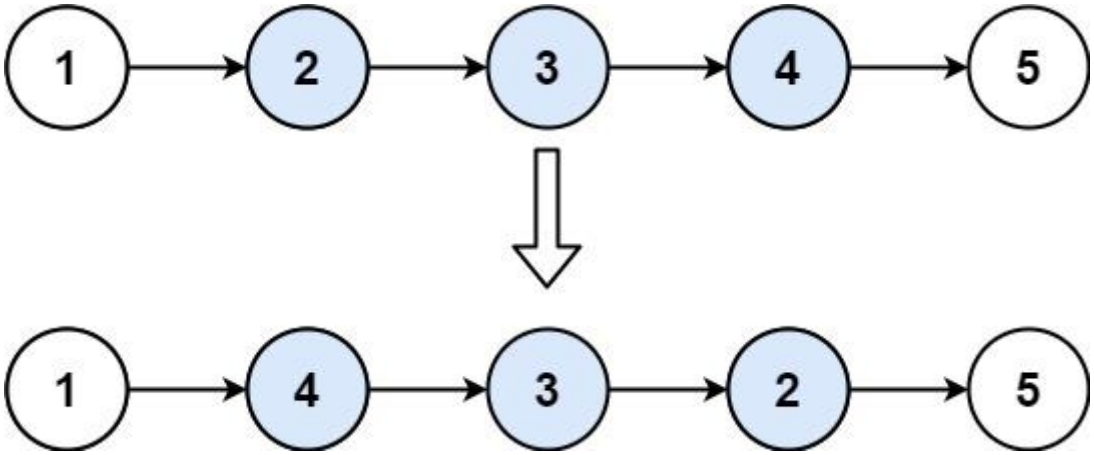
```
    ListNode* slow = head;
    ListNode* fast = head;
    while (fast->next != nullptr && fast->next->next != nullptr)
    {
        slow = slow->next;
        fast = fast->next->next;
    }

    // Step 2: Reverse second half
    ListNode* prev = nullptr;
    ListNode* curr = slow->next;
    while (curr != nullptr)
    {
        ListNode* nextNode = curr->next;
        curr->next = prev;
        prev = curr;
        curr = nextNode;
    }

    // Step 3: Compare first and second halves
    ListNode* first = head;
    ListNode* second = prev;  // head of reversed second half
    while (second != nullptr)
    {
        if (first->val != second->val)
        {
            return false;
        }
        first = first->next;
        second = second->next;
    }
    return true;
}
```

****

Q3. Given the head of a singly linked list and two integers left and right where left <= right, reverse the nodes of the list from position left to position right, and return the reversed list.
Example 1:



Input: head = [1,2,3,4,5], left = 2, right = 4
Output: [1,4,3,2,5]
Example 2:

Input: head = [5], left = 1, right = 1
Output: [5]

One of the solution:

```
ListNode* reverseBetween(ListNode* head, int left, int right) {
    if (head == nullptr || left == right) return head;

    // Step 1: Create a dummy node
    ListNode dummy(0);
    dummy.next = head;
    ListNode* prev = &dummy;

    // Step 2: Move `prev` just before the `left` node
    for (int i = 1; i < left; i++) {
        prev = prev->next;
    }

    // Step 3: Reverse from left to right
    ListNode* curr = prev->next;
    ListNode* nextNode = nullptr;
    ListNode* prevReversed = nullptr;
```

```
    for (int i = 0; i <= right - left; i++) {
        nextNode = curr->next;
        curr->next = prevReversed;
        prevReversed = curr;
        curr = nextNode;
    }

    // Step 4: Reconnect
    prev->next->next = curr;   // connect tail of reversed part to the rest
    prev->next = prevReversed; // connect prev to head of reversed part

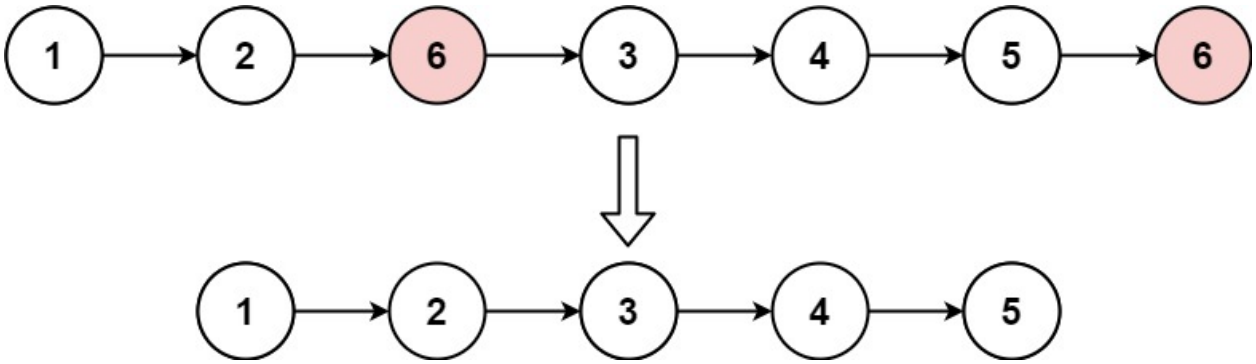
    return dummy.next;
}
```

****

Set-B

Q1. Given the head of a linked list and an integer key, remove all the nodes of the linked list that has Node.val == key, and return the new head.

Example:



Input: head = [1,2,6,3,4,5,6], val = 6
Output: [1,2,3,4,5]
Function declaration: ListNode* removeElements(ListNode* head, int val) {}

One of the solution:

```
ListNode* removeElements(ListNode* head, int val) {
    // Step 1: Create a dummy node before head
    ListNode dummy(0);
    dummy.next = head;
    ListNode* current = &dummy;

    // Step 2: Traverse and remove nodes
    while (current->next != nullptr) {
        if (current->next->val == val) {
            // delete the node by skipping it
            ListNode* temp = current->next;
            current->next = current->next->next;
            delete temp;  // free memory (optional, but good practice)
        } else {
            current = current->next;
        }
    }
```

```
    // Step 3: Return new head
    return dummy.next;
}
```

****

Q2. Given the head of a singly linked list, return true if it is a palindrome or false otherwise.
Example :
Input:



Output: true
Function declaration: bool isPalindrome(ListNode* head) { }

One of the solution:

```
bool isPalindrome(ListNode* head)
{
    if (head == nullptr || head->next == nullptr)
    {
        return true;  // Empty list or single node is always palindrome
    }

    // Step 1: Find middle using slow and fast pointers
    ListNode* slow = head;
    ListNode* fast = head;
    while (fast->next != nullptr && fast->next->next != nullptr)
    {
        slow = slow->next;
        fast = fast->next->next;
    }

    // Step 2: Reverse second half
```

```
    ListNode* prev = nullptr;
    ListNode* curr = slow->next;
    while (curr != nullptr)
    {
        ListNode* nextNode = curr->next;
        curr->next = prev;
        prev = curr;
        curr = nextNode;
    }

    // Step 3: Compare first and second halves
    ListNode* first = head;
    ListNode* second = prev;  // head of reversed second half
    while (second != nullptr)
    {
        if (first->val != second->val)
        {
            return false;
        }
        first = first->next;
        second = second->next;
    }
    return true;
}
```

****

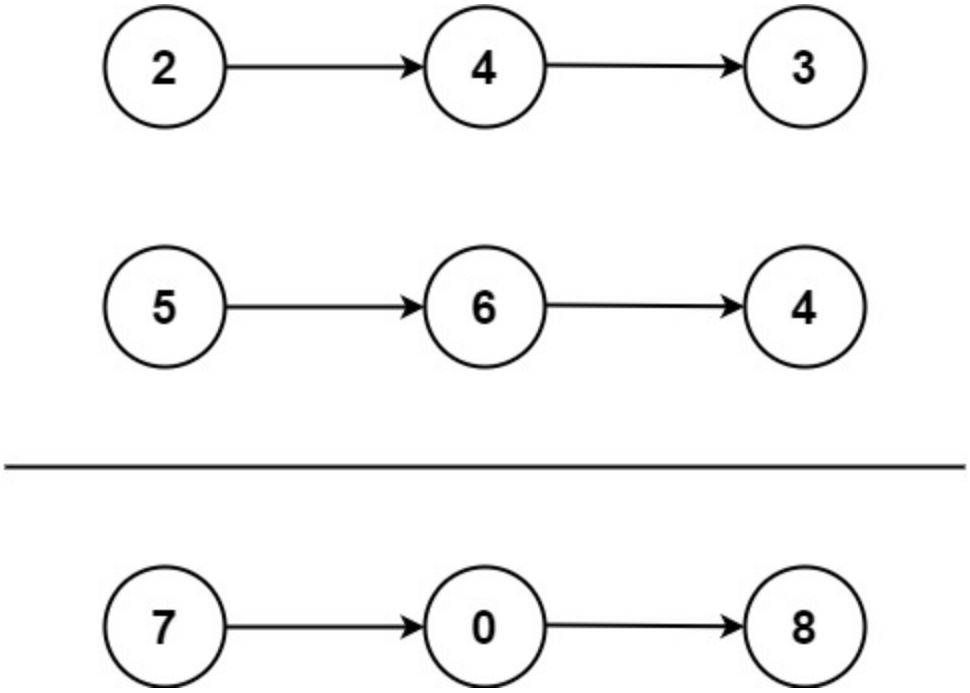
Q3. You are given two non-empty linked lists representing two non-negative integers. The digits are stored in reverse order, and each of their nodes contains a single digit. Add the two numbers and return the sum as a linked list.

You may assume the two numbers do not contain any leading zero, except the number 0 itself.

Example :

Input: l1 = [2,4,3], l2 = [5,6,4]

Output: [7,0,8]



Explanation: So the first linked list stores integer 342 in the reversed order as 243 and similarly, the second linked list stores integer 465 in the reversed order as 564, sum of 342 and 465 is 807, and stored in reversed order as 708 in a result linked list.
Function declaration: ListNode* addTwoNumbers(ListNode* l1, ListNode* l2) {}

One of the solution:

```
ListNode* addTwoNumbers(ListNode* l1, ListNode* l2) {
    ListNode dummy(0);  // dummy head
    ListNode* current = &dummy;
    int carry = 0;

    while (l1 != nullptr || l2 != nullptr || carry != 0) {
        int x = (l1 != nullptr) ? l1->val : 0;
        int y = (l2 != nullptr) ? l2->val : 0;

        int sum = x + y + carry;
        carry = sum / 10;

        current->next = new ListNode(sum % 10);
        current = current->next;
```

```
        if (l1 != nullptr) l1 = l1->next;
        if (l2 != nullptr) l2 = l2->next;
    }

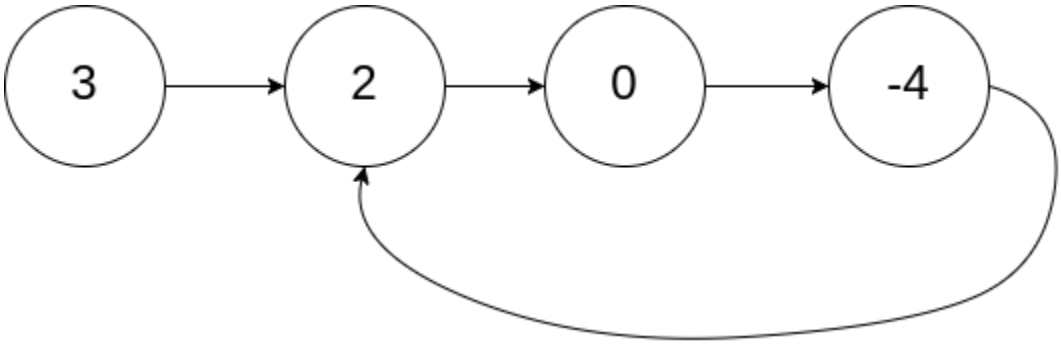
    return dummy.next;
}
```

****

Set-C

Q1. Given head, the head of a linked list, determine if the linked list has a cycle in it.
There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the next pointer. Internally, pos is used to denote the index of the node that tail's next pointer is connected to. Note that pos is not passed as a parameter. Return true if there is a cycle in the linked list. Otherwise, return false.

Example :



Input: head = [3,2,0,-4], pos = 1
Output: true

Explanation: There is a cycle in the linked list, where the tail connects to the 1st node (0-indexed).

One of the solution:

```
bool hasCycle(ListNode *head)
{
   if (head == nullptr || head->next == nullptr)
   {
      return false;  // Empty list or only one node → no cycle
   }

   ListNode *slow = head;
   ListNode *fast = head;

   while (fast != nullptr && fast->next != nullptr)
   {
```

```
        slow = slow->next;                  // move 1 step
        fast = fast->next->next;            // move 2 steps

        if (slow == fast)
        {
            return true; // cycle detected
        }
    }

    return false;                           // fast reached end → no cycle
}
```

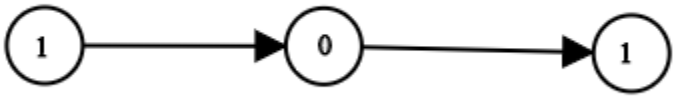
****

Q2. Given head which is a reference node to a singly-linked list. The value of each node in the linked list is either 0 or 1. The linked list holds the binary representation of a number. Return the decimal value of the number in the linked list.

The most significant bit is at the head of the linked list.

Example:

Input: [1,0,1]



Output: 5

Explanation: (101) in base 2 = (5) in base 10

Function declaration: int getDecimalValue(ListNode* head) {}

One of the solution:

```
int getDecimalValue(ListNode* head) {
    int result = 0;
    ListNode* current = head;
```

```
    while (current != nullptr) {
        result = result * 2 + current->val;  // shift left & add bit
        current = current->next;
    }

    return result;
}
```

****

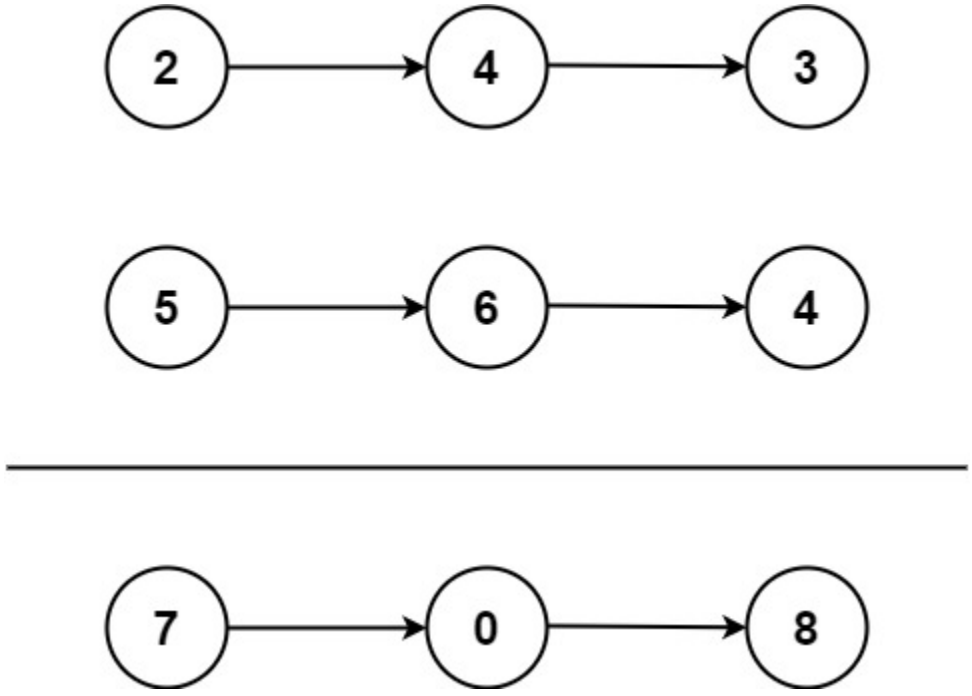
Q3. You are given two non-empty linked lists representing two non-negative integers. The digits are stored in reverse order, and each of their nodes contains a single digit. Add the two numbers and return the sum as a linked list.

You may assume the two numbers do not contain any leading zero, except the number 0 itself.

Example :

Input: l1 = [2,4,3], l2 = [5,6,4]

Output: [7,0,8]



Explanation: So the first linked list stores integer 342 in the reversed order as 243 and similarly, the second linked list stores integer 465 in the reversed order as 564, sum of 342 and 465 is 807, and stored in reversed order as 708 in a result linked list.

Function declaration: ListNode* addTwoNumbers(ListNode* l1, ListNode* l2) {}

One of the solution:

```
ListNode* addTwoNumbers(ListNode* l1, ListNode* l2) {
    ListNode dummy(0);  // dummy head
    ListNode* current = &dummy;
    int carry = 0;

    while (l1 != nullptr || l2 != nullptr || carry != 0) {
        int x = (l1 != nullptr) ? l1->val : 0;
        int y = (l2 != nullptr) ? l2->val : 0;

        int sum = x + y + carry;
        carry = sum / 10;

        current->next = new ListNode(sum % 10);
        current = current->next;

        if (l1 != nullptr) l1 = l1->next;
        if (l2 != nullptr) l2 = l2->next;
    }

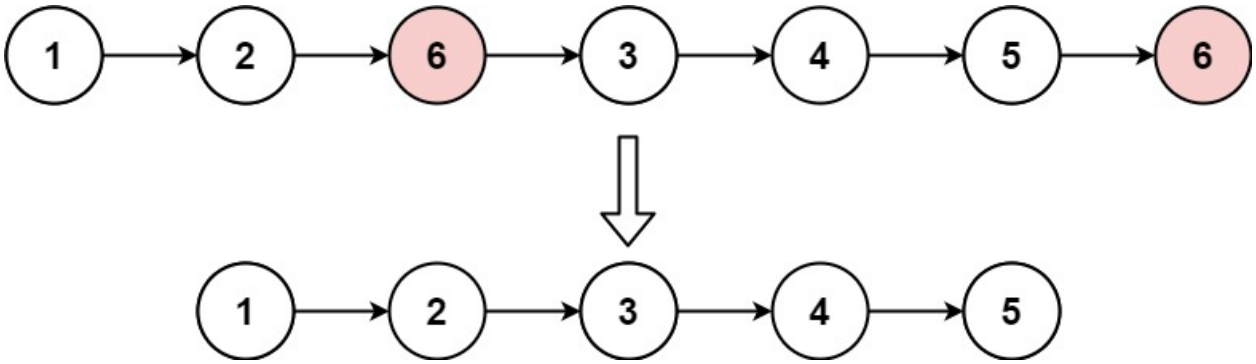
    return dummy.next;
}
```

****

Set-D

Q1. Given the head of a linked list and an integer key, remove all the nodes of the linked list that has Node.val == key, and return the new head.

Example:



Input: head = [1,2,6,3,4,5,6], val = 6
Output: [1,2,3,4,5]
Function declaration: ListNode* removeElements(ListNode* head, int val) {}

One of the solution:

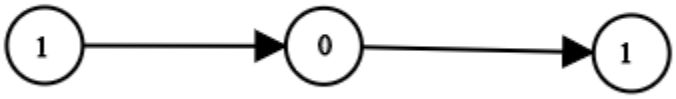
```
ListNode* removeElements(ListNode* head, int val) {
    // Step 1: Create a dummy node before head
    ListNode dummy(0);
    dummy.next = head;
    ListNode* current = &dummy;

    // Step 2: Traverse and remove nodes
    while (current->next != nullptr) {
        if (current->next->val == val) {
            // delete the node by skipping it
            ListNode* temp = current->next;
            current->next = current->next->next;
            delete temp;  // free memory (optional, but good practice)
        } else {
            current = current->next;
        }
    }
```

```
    // Step 3: Return new head
    return dummy.next;
}
```

****

Q2. Given head which is a reference node to a singly-linked list. The value of each node in the linked list is either 0 or 1. The linked list holds the binary representation of a number.
Return the decimal value of the number in the linked list.
The most significant bit is at the head of the linked list.
Example:
Input: [1,0,1]



Output: 5
Explanation: (101) in base 2 = (5) in base 10
Function declaration: int getDecimalValue(ListNode* head) {}

One of the solution:

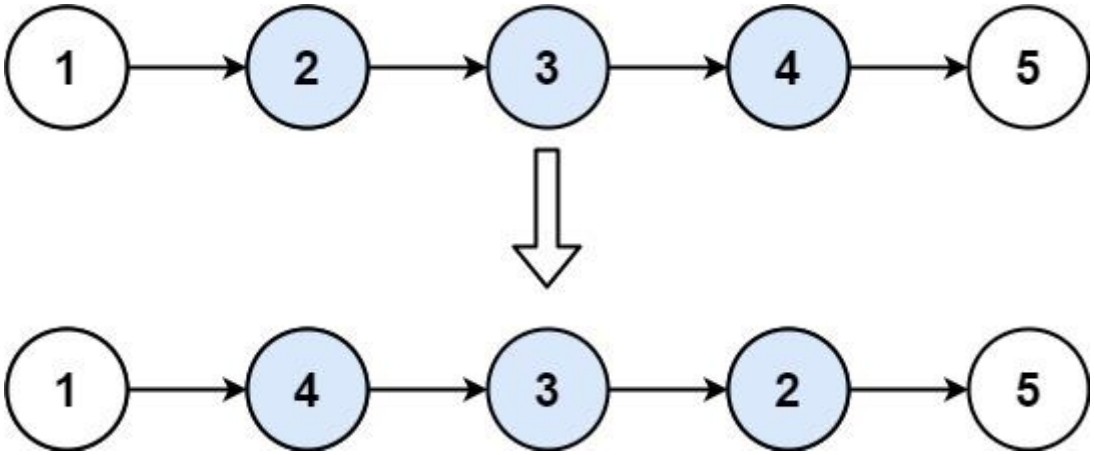
```
int getDecimalValue(ListNode* head) {
    int result = 0;
    ListNode* current = head;

    while (current != nullptr) {
        result = result * 2 + current->val;  // shift left & add bit
        current = current->next;
    }

    return result;
}
```

****

Q3. Given the head of a singly linked list and two integers left and right where left <= right, reverse the nodes of the list from position left to position right, and return the reversed list.
Example 1:



Input: head = [1,2,3,4,5], left = 2, right = 4
Output: [1,4,3,2,5]
Example 2:

Input: head = [5], left = 1, right = 1
Output: [5]

One of the solution:

```
ListNode* reverseBetween(ListNode* head, int left, int right) {
    if (head == nullptr || left == right) return head;

    // Step 1: Create a dummy node
    ListNode dummy(0);
    dummy.next = head;
    ListNode* prev = &dummy;

    // Step 2: Move `prev` just before the `left` node
    for (int i = 1; i < left; i++) {
        prev = prev->next;
    }

    // Step 3: Reverse from left to right
    ListNode* curr = prev->next;
    ListNode* nextNode = nullptr;
    ListNode* prevReversed = nullptr;
```

```
    for (int i = 0; i <= right - left; i++) {
        nextNode = curr->next;
        curr->next = prevReversed;
        prevReversed = curr;
        curr = nextNode;
    }

    // Step 4: Reconnect
    prev->next->next = curr;   // connect tail of reversed part to the rest
    prev->next = prevReversed; // connect prev to head of reversed part

    return dummy.next;
}
```

****